

Machine Learning Fundamentals

COSC 3430 Artificial Intelligence | Fall 26 | Augustana U.

Table of Contents

- [1. Why Machine Learning?](#)
- [2. What is Machine Learning?](#)
- [3. Machine learning applications and techniques](#)
- [4. Simple Example: Spam filter](#)
- [5. Types of Machine Learning](#)
- [6. Supervised Learning](#)
- [7. Unsupervised Learning](#)
- [8. Semi-supervised Learning](#)
- [9. Self-supervised Learning](#)
- [10. Reinforcement learning](#)
- [11. Batch vs. Online learning](#)
- [12. Online learning](#)
- [13. Instance-based vs. Model-based learning](#)
- [14. Extended Example: Does money make people happy? \(Difficult\)](#)
- [15. Extended Example: How can you diagnose illness? \(Simple\)](#)
- [16. PRACTICE Testing and improving a classification model](#)
- [17. The Machine Learning Workflow](#)
- [18. PRACTICE Fitting a linear model to data \(Simple\)](#)
- [19. Making Predictions](#)
- [20. Visualizing the Model \(Optional\)](#)
- [21. New! Interactive visualization of the model](#)
- [22. PRACTICE Fitting a linear model](#)
- [23. Key Points](#)
- [24. Next Topic](#)
- [25. Glossary](#)
- [26. Sources](#)

1. Why Machine Learning?

Modern computing systems increasingly operate in environments where:

- Data is abundant ("**Big Data**"¹)
- **Explicit** rules are difficult to specify
- **Adaptation** is required

Traditional programming follows the pattern²:

Rules (written by programmer) + Data → Output

Example: program after coding

```
if (email contains "win money!!!") -> spam
else -> not spam
```

Machine learning reverses this relationship³:

Data + Outputs → Rules (model) Rules + New Data → Output

Example: model after training⁴

```
if (learned_score(email) > threshold) -> spam  
else -> not spam
```

Applications:

- Email spam detection
- Credit card fraud detection
- Image classification

Remember:

- Traditionally, rules are explicitly written.
- In machine learning, rules are learned from data.

2. What is Machine Learning?

- Machine learning can be defined as:

Giving computers the ability to learn without being explicitly programmed.

- Or more specifically as:

Learning a function from data

- This involves four basic components:

1. **Features (inputs)** — measurable quantities (email sender)
2. **Target (output)** — what we want to predict (if email is spam or ham)
3. **Model** — a mathematical function mapping inputs to outputs (Naive Bayes)
4. **Loss function** — a measure of prediction error (Cross entropy)

- What's the goal?

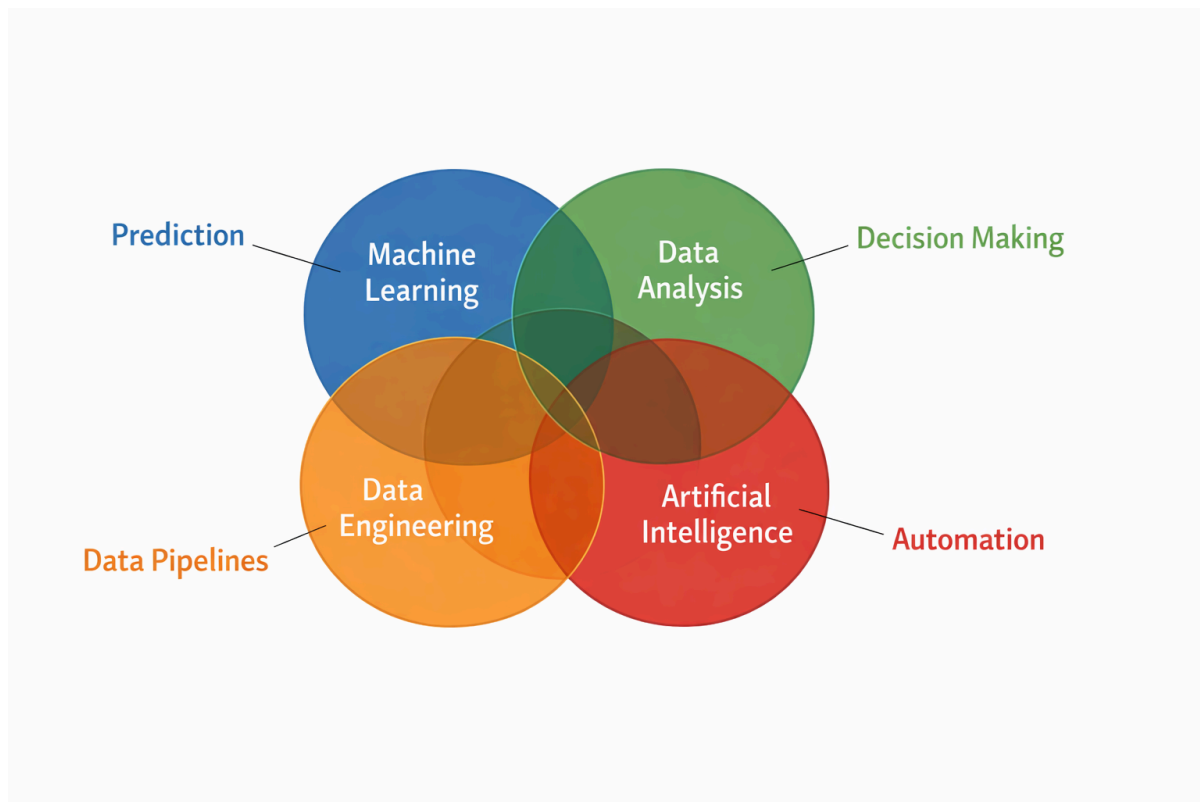
The goal is not to memorize the data, but to perform well on new, unseen data.

- What's a useful model?

A model that performs well on training data but poorly on new data is not useful.

ML is usually counted as a **subfield of artificial intelligence**, and it overlaps with neighbouring fields (see diagram):

- Data engineering (data pipelines)
- Data analysis (decision making)
- Artificial intelligence (automation)



3. Machine learning applications and techniques

- Can you think of useful applications for predictive machine learning?

APPLICATION	TECHNIQUE
Analyzing product images on a production line to classify them	Image classification with CNNs or transformers
Detecting tumors in brain scans	Semantic image segmentation with CNNs or transformers
Automatically classify news articles	NLP text classification with CNNs or transformers
Flag offensive comments on forums	NLP text classification
Summarizing long documents	NLP text
Creating a chatbot or assistant	NLP + QA modules, transformers
Forecasting company revenue from KPIs	Regression: linear, polynomial, SVM, random forest
Making apps react to voice	Speech recognition: CNNs,

APPLICATION	TECHNIQUE
commands	transformers, or RNNs
Detecting credit card fraud	Anomaly detection: isolation forest, GMM, autoencoders
Segmenting clients by purchases	Clustering: k-means, DBSCAN Grouping data based on distance
Visualizing high-dimensional data	Dimensionality reduction
Recommending products from past purchases	Recommender systems using ANN (trained on purchase patterns)
Building game-playing bots	Reinforcement learning (maximize a reward function)

4. Simple Example: Spam filter

A spam filter using traditional programming:

1. Examine what spam looks like. Identify patterns.
2. Write a detection algorithm for spam patterns.
3. Flag emails as spam if spam patterns are detected.
4. Test program until the algorithm is good enough.

A spam filter using machine learning:

1. Label a bunch of emails as spam.
2. Train the machine to identify spam. Create a model.
3. Test the model on new (unseen) data.
4. Improve model's ability to detect spam until it is good enough.

When a new spam pattern appears:

- The human algorithm has to be updated with a new rule.
- The ML model is updated by retraining on the new data.

How is the machine learning approach better?

- The ML model re-training can be automated when new data appear
- The program is much shorter, (more accurate,) and easier to maintain
- The ML approach scales to Big Data easily (many emails, much spam)
- Models can be used for data mining to improve understanding
- ML can be applied in fluctuating environments with big data

What's the bottleneck of machine learning?

- Data may be of **low quality** (noise, mislabeling)
- Example: legitimate emails are incorrectly labeled as spam (or vice versa), causing the model to learn wrong patterns.
- Data may be **insufficient** (during training)

- Example: only a small number of spam emails are available, so the model cannot learn reliable indicators of spam.
- Data may be **non-representative** (biased)
- Example: training data comes mostly from one source (e.g., corporate emails), so the model performs poorly on personal or international emails.
- Data may be **imbalanced**
- Example: 95% of emails are non-spam and only 5% are spam, leading the model to favor predicting “not spam.”
- Data may be **missing important features**
- Example: the dataset does not include sender reputation or email metadata, which are strong indicators of spam.
- Data may be **including irrelevant features**
- Example: features like email ID numbers or timestamps unrelated to spam are included, adding noise and confusing the model.

5. Types of Machine Learning

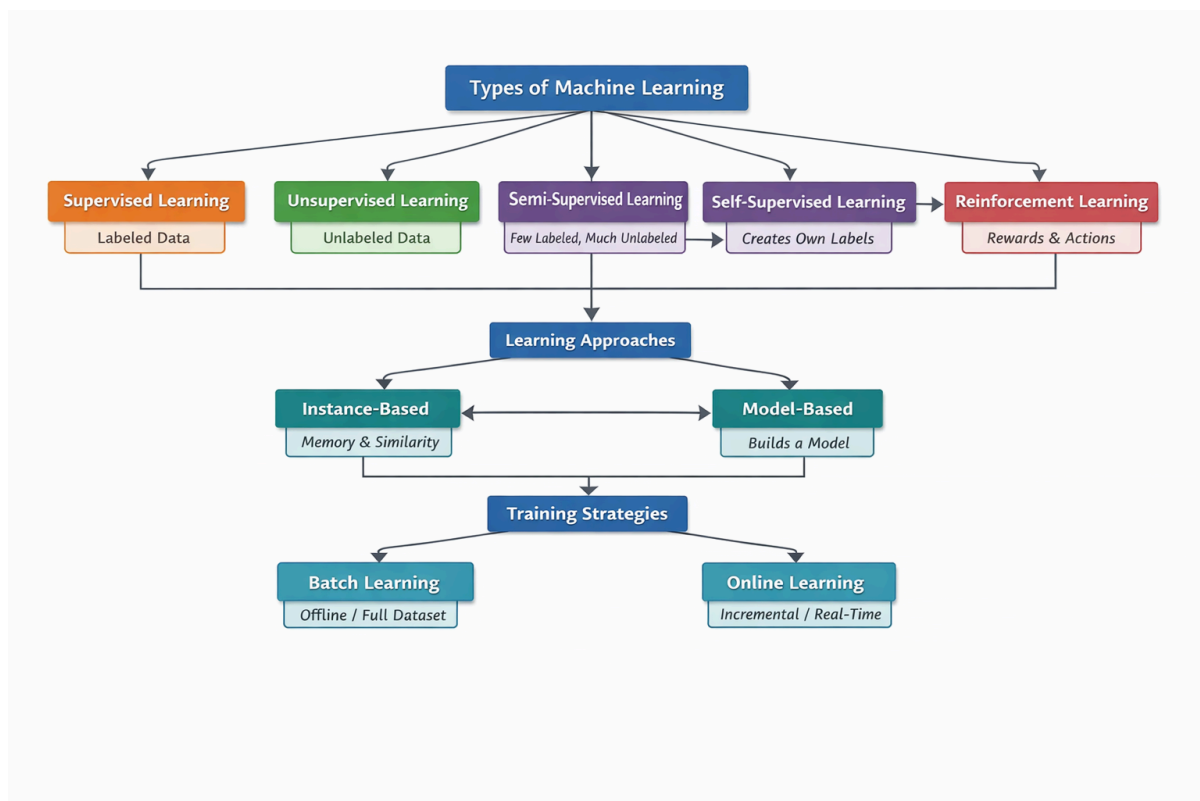


Figure 1: ML types and paradigms overview

We distinguish two main types of machine learning by the data they use:

1. Supervised learning (labeled data sets)
2. Unsupervised learning (unlabeled data sets)

Between and beyond these two lie several variants (semi-supervised, self-supervised, and reinforcement learning), described below.

6. Supervised Learning

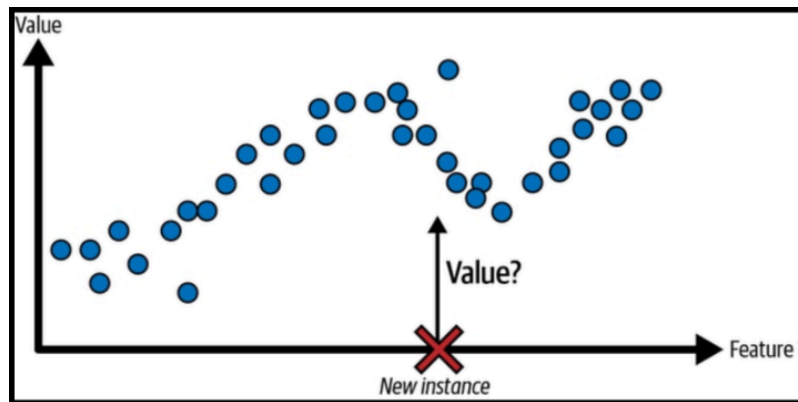


Figure 2: Regression - predict new value

- Data: includes both inputs and correct outputs
- Goal: learn mapping from inputs to outputs
- Price: data to learn from must be labeled

Examples:

- Predicting house prices (regression)
- Classifying emails as spam or not spam (Bayes)
- Diagnosing illness from symptoms (classification)

7. Unsupervised Learning

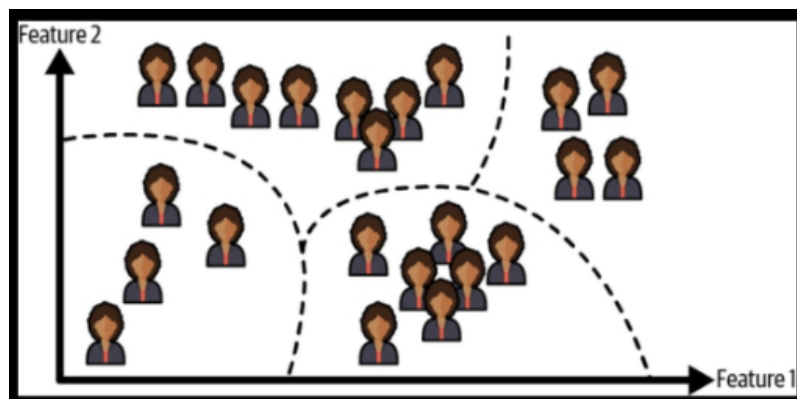
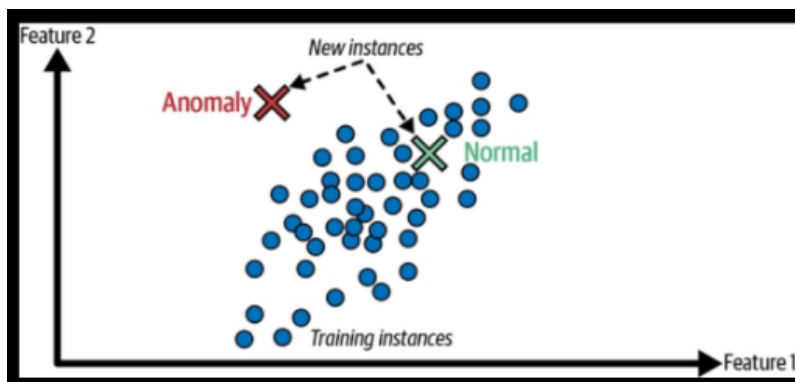


Figure 3: Clustering - group by features

- Data: includes inputs only
- Goal: discover structure in the data
- Price: model may be misled more easily (no ground truth)

Examples:

- Clustering customers (group by category)
- Dimensionality reduction (make problem simpler)
- Detecting anomalies (unusual transactions)



8. Semi-supervised Learning

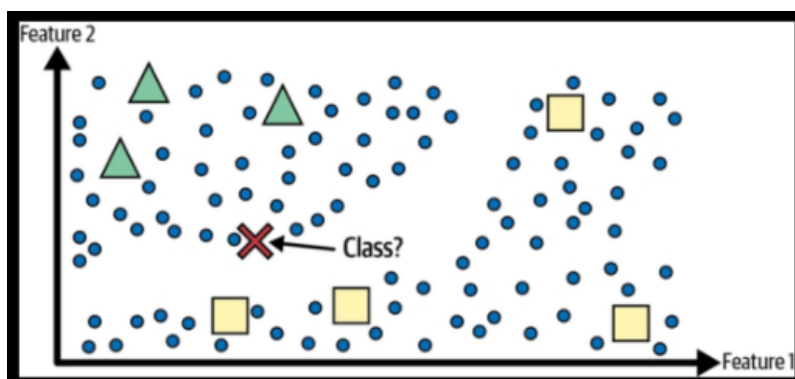


Figure 4: Unlabeled data help classify a new instance

- Data: small amount labeled, large amount unlabeled⁵
- Goal: leverage structure in unlabeled data to improve predictions
- Price: assumptions about data structure may be wrong

Examples:

- Classifying emails when only few are labeled spam/ham
- Image classification w/few labeled, many unlabeled images
- Speech recognition w/limited transcribed audio, large raw datasets

9. Self-supervised Learning

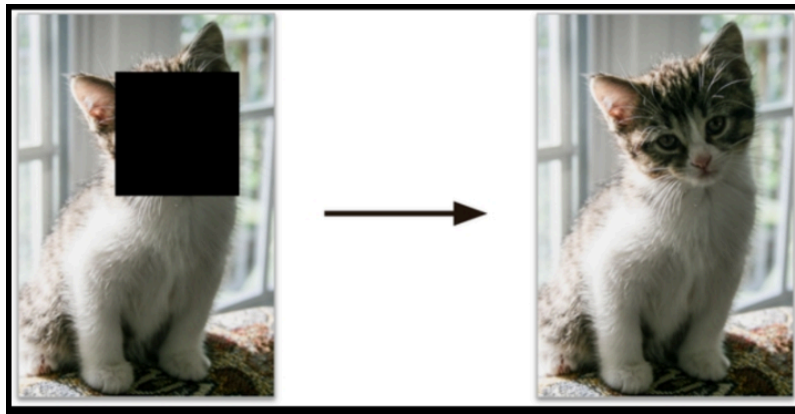


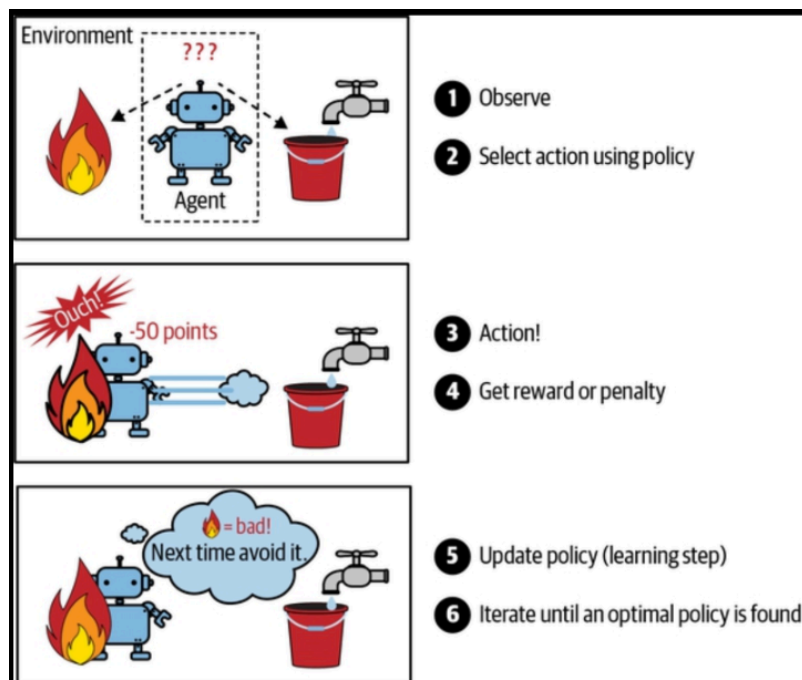
Figure 5: Masked input (left) and target image (right)

- Data: uses unlabeled data, creates its own labels from data⁶
- Goal: learn useful representations by solving auxiliary tasks
- Price: learned task may not align perfectly with final objective

Examples:

- Predicting missing words in a sentence (language models)
- Predicting next word/token in text (transformers)
- Filling in masked parts of an image (vision models)

10. Reinforcement learning



- Data: generated through interaction with environment
- Goal: learn policy that maximizes long-term reward
- Price: exploration costly, learning unstable, slow

Examples:

- Training a game-playing agent (chess)
- Robotics control (learning to walk)
- Optimizing recommendations through user interaction

11. Batch vs. Online learning

Batch Learning	Online Learning
Data: full dataset upfront	Data: sequential / stream
Goal: train on all data	Goal: update continuously
Price: costly retraining	Price: noise sensitivity
Examples: nightly spam training with regression	Example: real-time spam filtering with live recommendation updates

12. Online learning

The model is updated incrementally, one data point (or small batch) at a time, as new data arrives.

If the data does not fit in main memory, only part of the data is loaded, runs a training step then loads the next part (out-of-core learning, done incrementally, in practice often off-line).

Important parameter: **learning rate** = how fast should the model adapt to changing data. High rate adapts quickly but forgets old data too. Low rate leads to more inertia (slower learning) and less sensitivity to noise in new data or to outliers.

13. Instance-based vs. Model-based learning

- This relates to the ability of the models to **generalize** = making good predictions for examples it has never seen before. Two approaches dominate:

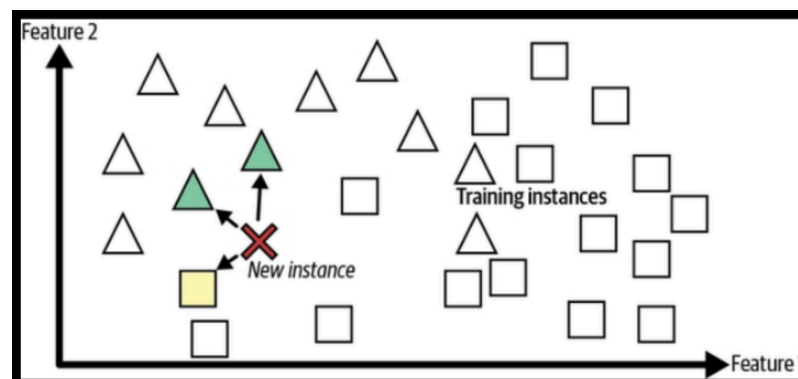


Figure 6: New instance is a triangle because majority of similar instances are

For instance-based learning, the system learns the examples by heart, then generalizes to new cases using a **similarity measure** to compare them to the learned examples.

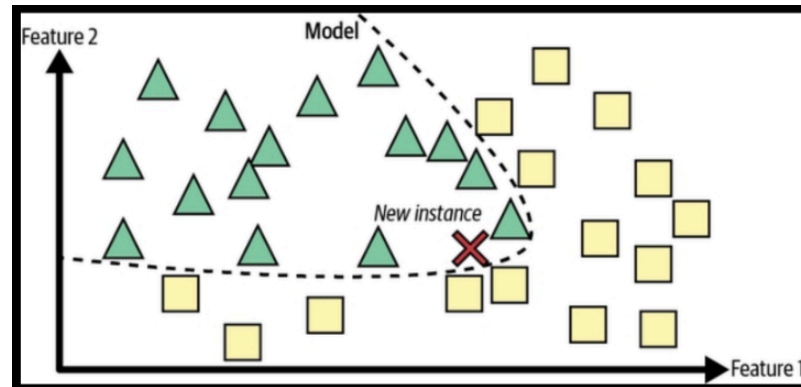


Figure 7: Build a model to make predictions of new data

In model-based learning, a model is built from the training examples (by learning its parameters), and that model is then used to make predictions on new instances.

- Example: Predicting a house's price from its size
 1. **Model-based:** Fit a line through all the (size,price) points you've seen: $\text{price} = a \times \text{size} + b$. For a new house, plug its size into that formula.
 2. **Instance-based:** Don't fit anything. For a new house, find the 3 most similar-size houses you've seen, and average their prices to get the wanted price.

14. Extended Example: Does money make people happy? (Difficult)

Géron uses a rather difficult example. You can find it with code in a [Colab notebook](#) - you can run the code cells in the notebook.

1. Download data: Better Life Index from OECD/World Bank
2. Join tables and sort by GDP per capita ("money")
3. Visualize the data
4. Select a linear model (because of the visualization)
5. Train the model (use Python library)
6. Make a prediction
7. Pick a different model

Instead, we will use two simpler examples - a decision tree and a linear regression (trendline fit)

15. Extended Example: How can you diagnose illness? (Simple)

Consider diagnosing an illness based on symptoms.

- Input (features): symptoms (e.g., fever, cough)

- Output (target): diagnosis (e.g., flu, cold)
- Model: a decision rule (or tree)

The model attempts to approximate the relationship:

$$\text{diagnosis} \approx f(\text{symptoms})$$

Process:

1. Build a **classification** model from **known** data instances
2. Test model to **classify** newly presented **unknown** data instances
3. Translate model into algorithmic **production** rules

Building a model from training data

- Dataset: hypothetical training data for a disease diagnosis

Patient ID#	Sore Throat	Fever	Swollen Glands	Congestion	Headache	Diagnosis
1	Yes	Yes	Yes	Yes	Yes	Strep throat
2	No	No	No	Yes	Yes	Allergy
3	Yes	Yes	No	Yes	No	Cold
4	Yes	No	Yes	No	No	Strep throat
5	No	Yes	No	Yes	No	Cold
6	No	No	No	Yes	No	Allergy
7	No	No	Yes	No	No	Strep throat
8	Yes	No	No	Yes	Yes	Allergy
9	No	Yes	No	Yes	Yes	Cold
10	Yes	Yes	No	Yes	Yes	Cold

Figure 8: Hypothetical disease diagnosis training data (Source: Roiger, 2021)

- **Observation:** Patient 1 has a sore throat, fever, swollen glands, is congested and has a headache. He was diagnosed with strep throat.
- A **decision tree** can be used to generalize a set of input instances as shown and transform it into rules.
- To **generalize**, we must make assumptions about the relative importance of attributes and their relationship.
- For example:
 - If a patient has swollen glands, the diagnosis is strep throat
 - If a patient does not have swollen glands and a fever, it's a cold
 - If a patient does not have swollen glands nor a fever, it's allergy

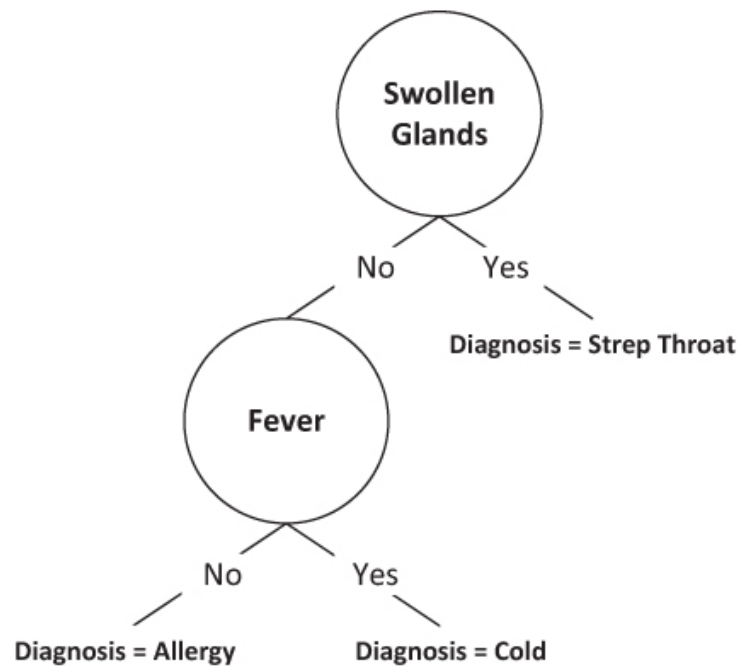


Figure 9: Decision tree for disease diagnosis (Source: Roiger, 2021)

- Notice that the attributes *sore throat*, *congestion* and *headache* do not enter our diagnostic prediction.

Testing the model on unknown instances

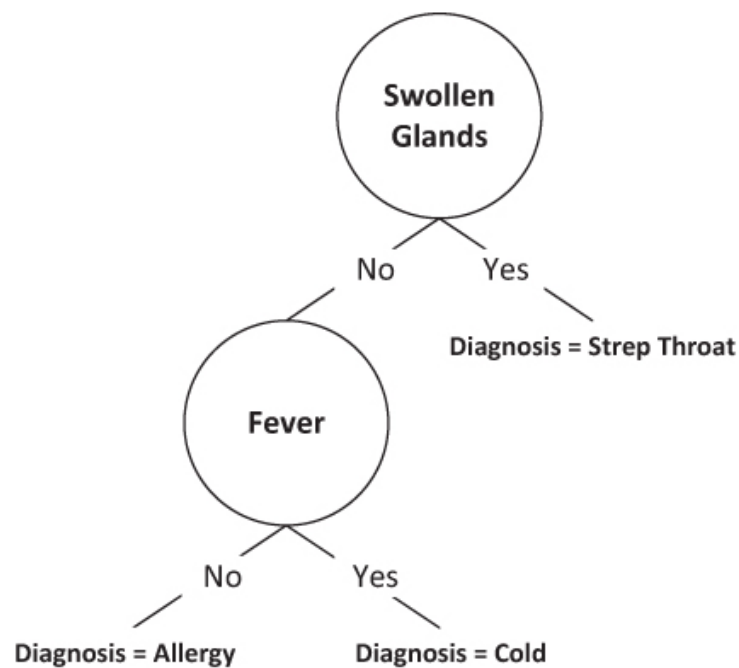


Figure 10: Decision tree for disease diagnosis

- Moving on to a new data set with unknown classification, i.e. no diagnosis
- Use the decision tree to classify the first two instances:

Patient ID	Sore Throat	Fever	Swollen Glands	Congestion	Headache	Diagnosis
11	No	No	Yes	Yes	Yes	?
12	Yes	Yes	No	No	Yes	?
13	No	No	No	No	Yes	?

Figure 11: Test data for disease diagnosis model (source: Roiger, 2021)

- Patient 11 has swollen glands but no fever => strep throat
- Patient 12 has no swollen glands but fever => cold

Translate model into production rules

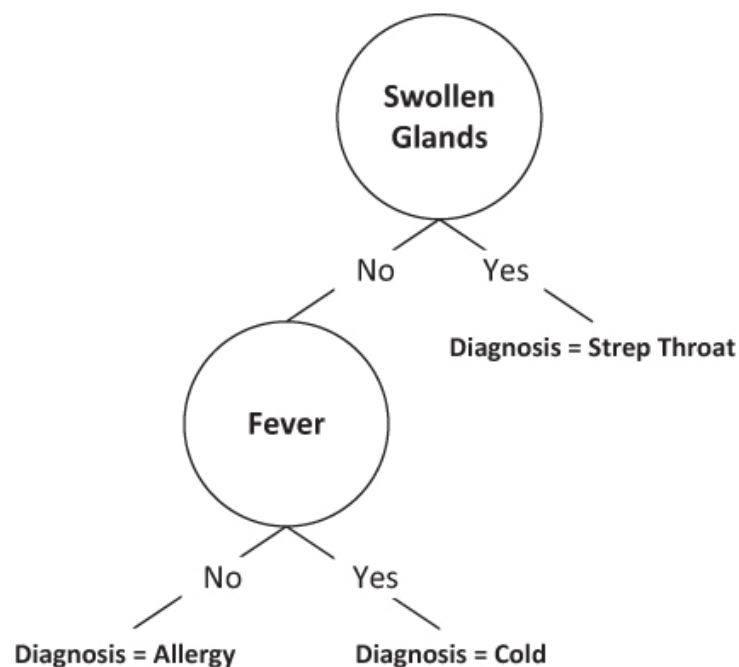


Figure 12: Decision tree for disease diagnosis (source: Roiger, 2021)

- General form of a *production rule* looks like pseudocode⁷:

IF antecedent condition
THEN consequent conditions

- The three *production rules* for the decision tree:

```
IF swollen glands = YES
THEN diagnosis = strep throat

IF swollen glands = No & Fever = Yes
THEN diagnosis = cold

IF swollen glands = No & Fever = No
THEN diagnosis = allergy
```

- Testing the rules on patient 13 yields: diagnosis = allergy

Patient ID	Sore Throat	Fever	Swollen Glands	Congestion	Headache	Diagnosis
11	No	No	Yes	Yes	Yes	?
12	Yes	Yes	No	No	Yes	?
13	No	No	No	No	Yes	?

Figure 13: Test data for disease diagnosis model (source: Roiger, 2021)

16. **PRACTICE** Testing and improving a classification model

This practice exercise consists of three parts:

1. Test the model on new patients using the diagnostic table
2. Modify the production rules by adding a new attribute
3. Draw the updated decision tree model

Handout (submit solution for points) + solution sheet

17. The Machine Learning Workflow

A typical workflow consists of the following steps:

1. Define the problem
2. Collect data
3. Prepare data
4. Choose a model
5. Train the model (includes splitting train/test data)
6. Evaluate the model
7. Use the model for prediction (production)

This process is iterative. At any stage, it may be necessary to revisit earlier steps.

Common Issues

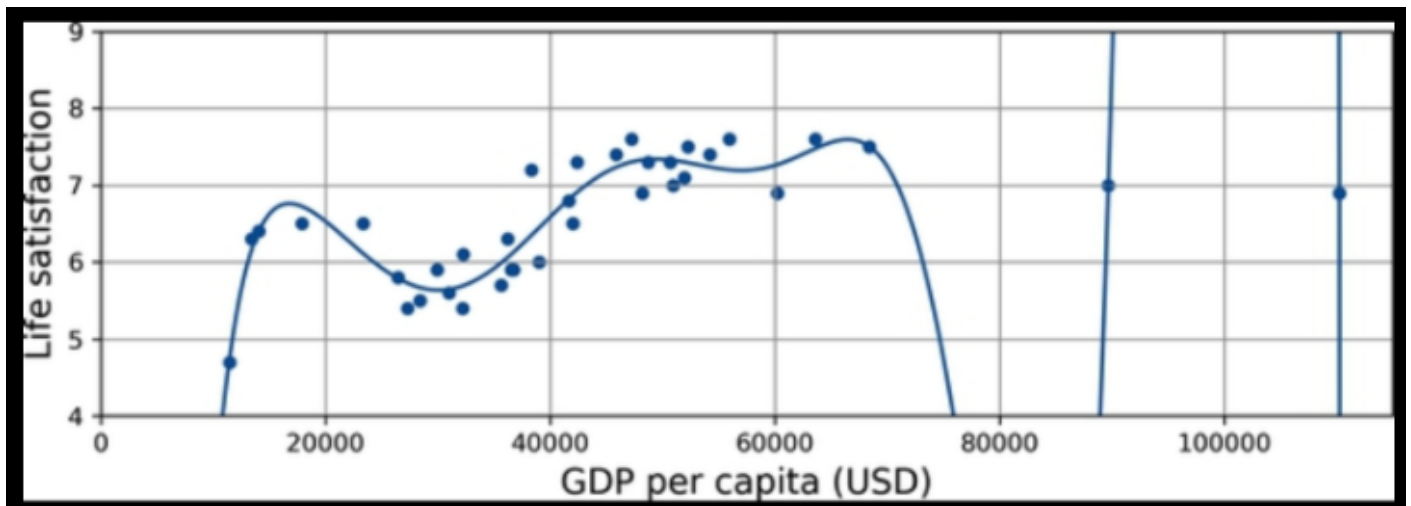


Figure 14: Overfitting the training data

- Overfitting: model learns noise instead of pattern
- Underfitting: model is too simple to capture structure

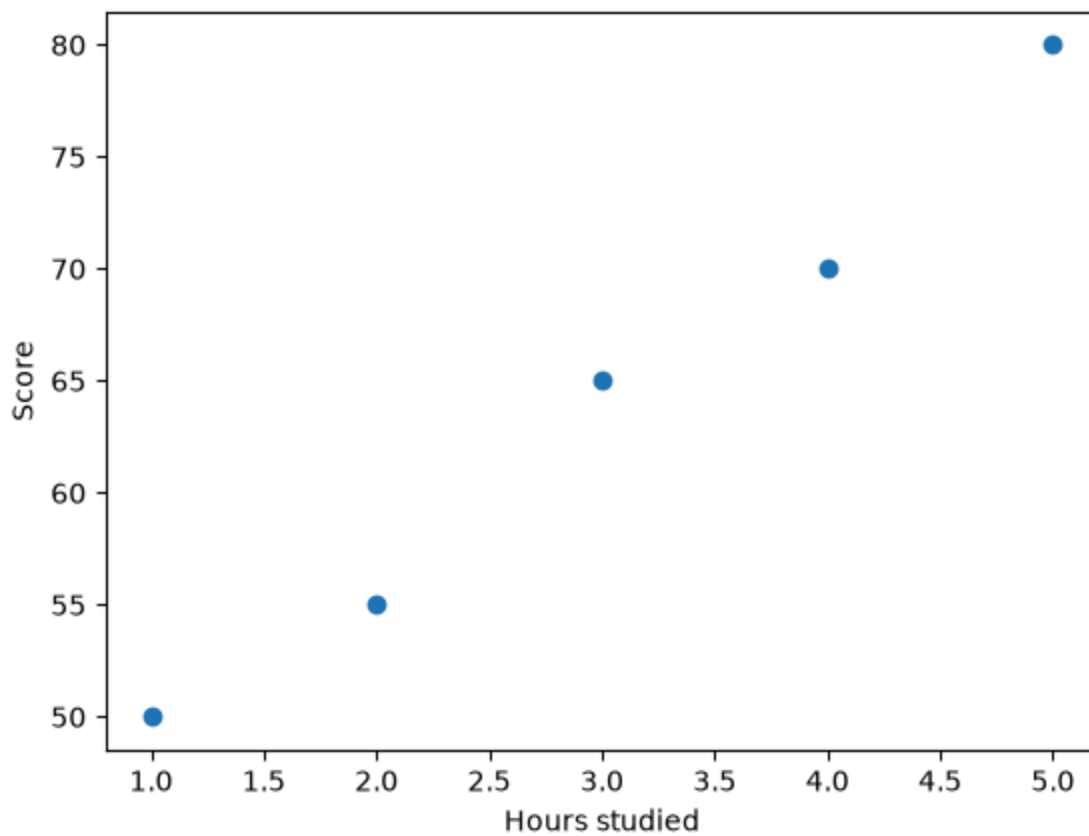
18. **PRACTICE** Fitting a linear model to data (Simple)

Open the colab notebook and code along: tinyurl.com/3430-linear-regression

- Data visualization without modeling

```
import matplotlib.pyplot as plt
import numpy as np
hours = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)
score = np.array([50, 55, 65, 70, 80])

plt.scatter(hours, score)
plt.xlabel("Hours studied")
plt.ylabel("Score")
plt.savefig("hours_score.png")
```



- The reshape function reshapes an ndarray without changing data:

```
import numpy as np

foo = np.array([1,2,3,4])
print(foo)
print(foo.shape)
print(foo.reshape(-1,1).shape)
```

```
[1 2 3 4]
(4,)
(4, 1)
```

- The following example fits a linear model to data.

```
import numpy as np
from sklearn.linear_model import LinearRegression

# Data
hours = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)
score = np.array([50, 55, 65, 70, 80])

# Model
```

```

model = LinearRegression()
model.fit(hours, score)

# Parameters
print("Slope:", model.coef_[0])
print("Intercept:", model.intercept_)

```

```

Slope: 7.5000000000000001
Intercept: 41.5

```

- Code explanation:

1. We import the numerical math library, numpy, and alias it as np, and we import the LinearRegression class from the machine learning library scikit-learn.
2. We create two arrays, hours and score.
3. Many ML libraries expect the input feature matrix X in 2D form. reshape changes the shape of the array without changing the data:

```

print(np.array([1, 2, 3, 4, 5]).shape) # 1D array with shape (5,)
print(hours.shape)                    # 2D array: 5 rows, 1 column

```

```

(5,)
(5, 1)

```

4. An empty model instance is created by LinearRegression().
5. model.fit(hours, score) trains the model: X is the input (hours), and y is the output (score). The method solves an optimization problem: it finds the values of β_0 and β_1 that minimize the sum of squared residuals (the sum of error squared):

$$\sum_{i=1}^n (y_i - (\beta_0 + \beta_1 x_i))^2$$

6. After .fit(), model.coef_ (β_1) and model.intercept_ (β_0) are available. They define the learned function: prediction = intercept + slope * x.
7. Numeric example: hours[2] = 3 and score[2] = 65. For x=3, the model predicts

$$\hat{y} = 41.5 + 7.5 \cdot 3 = 64.0$$

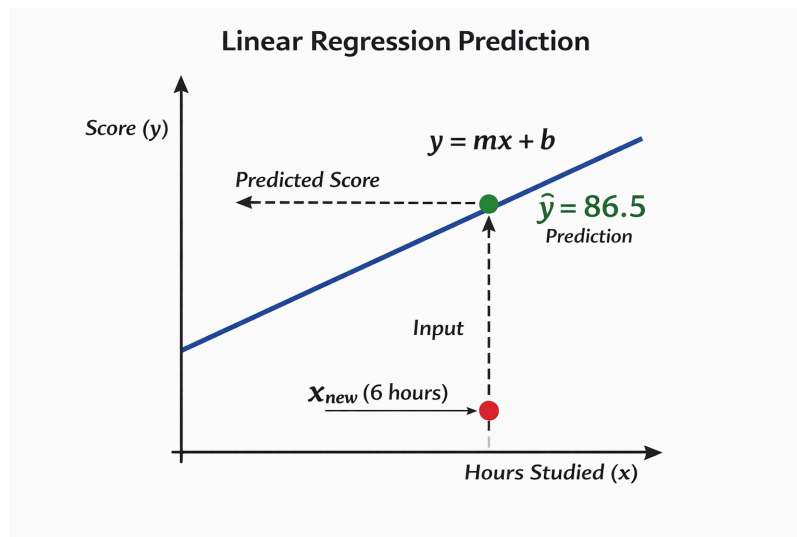
so the residual is

$$y - \hat{y} = 65 - 64 = 1.0$$

There are five such data equations but only two unknown parameters, so this is an **overdetermined system**.

8. The prediction answers the question: If I study x hours, what score does the model predict?
9. Each data point acts like a **constraint**. Since there are more constraints than unknowns, they usually cannot all be satisfied exactly, so we find the line that is closest overall in the least-squares sense.
10. There are many different possible loss functions, not just least squares. Which is best depends on the problem distribution.

19. Making Predictions



- The trained model can be used to make predictions:

```
print(model.predict([[6]]))
```

```
[86.5]
```

- This returns the predicted score for 6 hours of study.
- Code explanation:
 1. After training, the model has learned parameters β_0 (intercept) and β_1 (slope).
 2. The method `model.predict(X_new)` uses these parameters to compute predicted outputs for new input data.
 3. Internally, it applies the learned function:

$$\hat{y} = \beta_0 + \beta_1 x$$

to each input value.

4. The argument to `predict` must have the same **shape** as the training input X , i.e., a 2D array of shape $(n_samples, n_features)$.
5. Since we trained with one feature (hours), the model expects input of shape $(?, 1)$.

6. The input `[[6]]` is therefore:

- one sample (outer list)
- one feature (inner list)

so its shape is $(1, 1)$.

7. Using `[6]` instead would create a 1D array with shape $(1,)$, which is not accepted by `scikit-learn` for prediction.

8. The call `model.predict([[6]])` returns an array:

```
array([86.5])
```

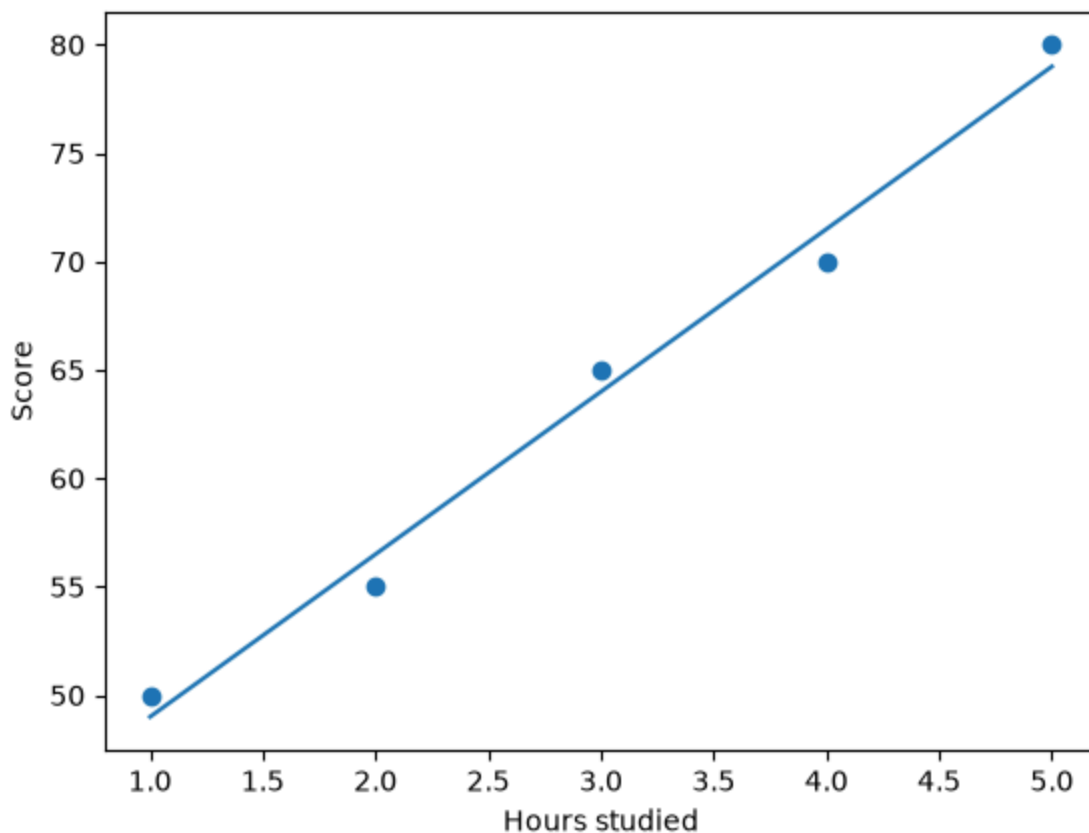
because predictions are always returned as a sequence (even for a single input).

9. Interpretation: for $x = 6$ hours, the model predicts a score of $\hat{y} = 86.5$.

20. Visualizing the Model (Optional)

```
import matplotlib.pyplot as plt

plt.scatter(hours, score)
plt.plot(hours, model.predict(hours))
plt.xlabel("Hours studied")
plt.ylabel("Score")
plt.savefig("../img/grade.png")
```



The line represents the learned relationship between input X (hours) and output Y (score).

21. New! Interactive visualization of the model

In a browser, open birkenkrahe.github.io/ml_demos/linear-regression.html

You can drag the red point or the slider to pick a new data point.

You can watch the model read the prediction straight off the line.

22. **PRACTICE** Fitting a linear model

Use the following data:

```
import numpy as np

temp = np.array([60, 65, 70, 75, 80]).reshape(-1, 1)
sales = np.array([30, 45, 50, 65, 80])
```

Tasks:

1. Fit a linear model
2. Predict the value at 85 F
3. Plot the data and fitted line
4. In one complete sentence at the end, explain what the fitted line tells us about the relationship between temperature and ice cream sales.

Use Gemini in Colab to complete these tasks:

- Create code cell and click on generate
- In the Gemini console, change to Gemini 3 Flash

Solution

- Sample prompt:

"Can you complete the tasks for me? Comment amply in the code to help me understand what is going on."

- Response:

"I have generated the code to perform the tasks: fit a linear model, predict sales at 85°F, and visualize the data and the fitted line. The fitted line shows that as the temperature increases, ice cream sales also tend to increase, indicating a positive linear relationship between the two variables."

- Code with comments:

```
import numpy as np
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt

# --- Task 1: Fit a linear model ---

# Re-using the data from the previous cell
# temp = np.array([60, 65, 70, 75, 80]).reshape(-1, 1)
# sales = np.array([30, 45, 50, 65, 80])

# Create a Linear Regression model instance
```

```

model_sales = LinearRegression()

# Fit the model to the 'temp' (independent variable) and 'sales' (dependent variable)
# The .fit() method learns the relationship (slope and intercept) between the input and output
model_sales.fit(temp, sales)

# Print the learned parameters (slope and intercept) for understanding the model
print("Slope (coefficient):", model_sales.coef_[0])
print("Intercept:", model_sales.intercept_)

# --- Task 2: Predict the value at 85°F ---

# Create a numpy array for the temperature at which to make a prediction (85°F)
# It needs to be reshaped to a 2D array as expected by the predict method
temp_to_predict = np.array([[85]])

# Use the trained model to predict the sales for 85°F
predicted_sales_85 = model_sales.predict(temp_to_predict)
print(f"Predicted sales at 85°F: {predicted_sales_85[0]:.2f}")

# --- Task 3 & 4: Plot the data and fitted line ---

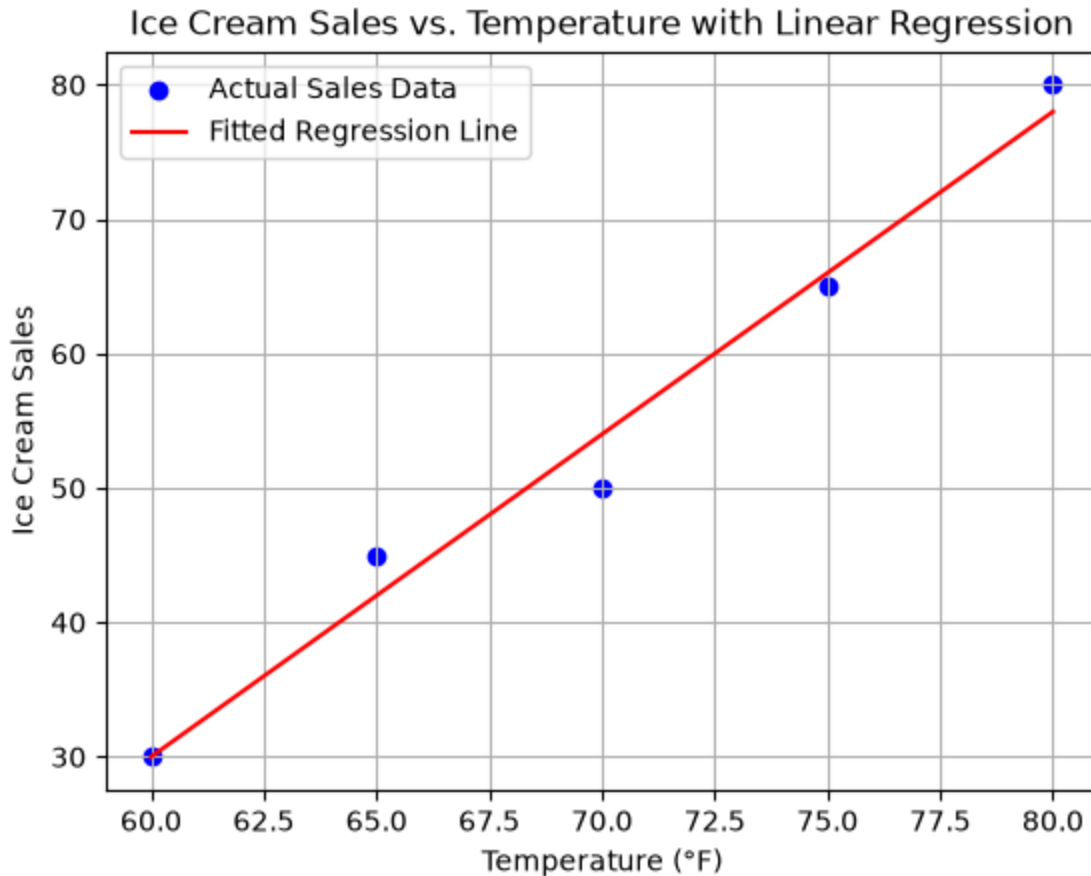
# Create a scatter plot of the original data points
plt.scatter(temp, sales, color='blue', label='Actual Sales Data')

# Plot the regression line
# To get the line, predict sales for the range of existing temperatures
plt.plot(temp, model_sales.predict(temp), color='red', label='Fitted Regression Line')

# Add labels and title for clarity
plt.xlabel("Temperature (°F)")
plt.ylabel("Ice Cream Sales")
plt.title("Ice Cream Sales vs. Temperature with Linear Regression")
plt.legend()
plt.grid(True)
plt.savefig("linear_regression2.png")

```





- Answer to the question:

The prediction at 85° F represents the model's estimate of how many ice cream units will be sold at that temperature, based on the linear pattern learned from the data.

23. Key Points

- Machine learning learns patterns from data
- A model is a function that maps inputs (X) to outputs (y)
- Models are learned by minimizing error on training data
- Predictions are estimates, not exact values
- The workflow is iterative
- Models should generalize to new data
- Overfitting and underfitting affect model performance

24. Next Topic

In this lecture, we learned how to predict numerical values using a linear model.

We moved from data → model → prediction.

Next, we will study a different kind of prediction:

How can we decide between categories instead of predicting numbers?

We will introduce **classification**:

- Predicting labels such as pass/fail or spam/not spam
- Understanding how models make decisions
- Using similarity between data points (k-nearest neighbors)

Instead of asking “What value?”, we will ask “Which class?”

We will also see how changing model parameters affects decisions, introducing the idea of model complexity.

25. Glossary

Basic concepts

Term	Definition
machine learning	Learning a function from data, not from written rules
Big Data	Data of high volume, velocity, variety, veracity, value
feature	A measurable input quantity, e.g. email sender (attribute)
target	The output we want to predict, e.g. spam or ham
label	The known correct target value of a data instance
instance	One data example: the feature values of one case
model	A function mapping inputs (X) to outputs (y)
parameter	A value the model learns from data, e.g. a slope
loss function	A measure of prediction error, e.g. cross entropy
training	Learning the model's parameters from data
inference	Using the trained model to predict new data
generalization	Predicting well on examples never seen before
ground truth	The known correct answer, used as a reference
prediction	The model's estimate for new input; not exact

Types of learning

Term	Definition
supervised learning	Learns from labeled data: inputs and correct outputs
unsupervised learning	Learns from unlabeled data; discovers structure
semi-supervised learning	A little labeled data, much unlabeled data
self-supervised learning	Creates its own labels from unlabeled data

Term	Definition
reinforcement learning	Learns by interaction to maximize long-term reward
policy	The learned rule for choosing an action (reinforcement)
reward	The value the learner tries to maximize over time
batch learning	Trains on the full data set at once
online learning	Updates the model step by step as data arrives
out-of-core learning	Trains on data too big for memory, part by part
learning rate	How fast the model adapts to changing data
instance-based learning	Compares new cases to stored examples by similarity
model-based learning	Builds a model from examples, then predicts with it
similarity measure	A way to compare a new instance with known ones

Tasks and techniques

Term	Definition
classification	Predicting a category (label), e.g. spam or ham
regression	Predicting a number, e.g. a house price
clustering	Grouping instances by similar features
dimensionality reduction	Reducing the number of features to simplify a problem
anomaly detection	Finding unusual instances, e.g. fraud
decision tree	A tree of attribute tests that classifies instances
production rule	IF condition THEN conclusion; a tree turned into rules
pseudocode	An algorithm in plain language, without syntax rules
linear regression	Fitting a straight line to data

Data problems and model quality

Term	Definition
noise	Random errors or mislabeled data
non-representative data	Training data unlike the data met in use
imbalanced data	Some classes far more common than others
overfitting	The model learns noise instead of the pattern
underfitting	The model is too simple to capture the structure
train/test split	Data set aside to test the trained model

Term	Definition
workflow	Define, collect, prepare, choose, train, evaluate, predict

Linear model

Term	Definition
slope	Change in the prediction per unit of input, β_1
intercept	The prediction when the input is 0, β_0
residual	Actual minus predicted value, $y - \hat{y}$
least squares	Choosing the line with the smallest sum of squared residuals
overdetermined system	More equations (data points) than unknowns
constraint	A condition each data point puts on the parameters

Code

Term	Definition
numpy	Python library for numerical arrays
ndarray	The numpy array type
shape	An array's dimensions, e.g. (5, 1)
reshape	Changes an array's shape, not its data
scikit-learn	Python machine learning library
LinearRegression	scikit-learn class for a linear model
.fit(X, y)	Trains the model: X = inputs, y = outputs
.predict(X)	Returns the model's predictions for inputs X
coef_	The learned slope(s) of a fitted model
intercept_	The learned intercept of a fitted model
matplotlib	Python plotting library (scatter, plot)

26. Sources

Most images: A. Géron, Hands-On Machine Learning with Scikit-Learn and PyTorch (O'Reilly, 2026). Decision-tree diagnosis images (patient data, decision tree, test data): R. Roiger, Data Mining: A Tutorial-Based Primer, 2e (2021).

Examples and code developed in collaboration with ChatGPT.

Footnotes:

¹ 5V definition: Big Data are characterized by high volume, velocity, variety, veracity and value.

² Humans make the rules, machines execute them. This requires the ability to foresee everything that could possibly go wrong - which is impossible in a complex environment.

³ This is not a reversal of the direction but about where the rules come from.

⁴ After training, the model behaves as if it had rules, which however were never written by a human. This is called the **inference** phase, which is preceded by a **training** phase: `model=train(emails, labels)` where `labels` label emails spam/not spam.

⁵ Example Google Photos: Once you upload all family photos, it automatically recognizes that A shows up in photos 1, 5, 11 while B shows up in 2, 5, 7 (unsupervised clustering). Label A and B and the service recognizes A and B in all photos.

⁶ The model learns by predicting missing parts of the input, e.g. reconstructing a masked region of an image, using the surrounding context as its own supervision.

⁷ This is exactly what pseudocode is: natural language without the constraints of syntactical rules. What used to be helpful in the past could in the future well become the standard for programming.

Author: Marcus Birkenkrahe

Created: 2026-09-19 Sat 22:00